

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.Doi Number

Natural Language Generation using Dependency Tree Decoding for Spoken Dialog Systems

Youngmin Park¹, and Sangwoo Kang²

¹Robotics Team, Hyundai Motor Company, Gyeonggi-do 16082, Korea

²Department of Software, Gachon University, Gyeonggi-do 13120, Korea

Corresponding author: Sangwoo Kang (e-mail: swkang@gachon.ac.kr).

This work was supported by the Basic Science Research Program through the National Research Foundation of Korea (NRF), funded by the Ministry of Science and ICT (No. NRF-2016R1C1B1014573).

ABSTRACT In this paper, we propose a new natural language generation method for spoken dialog systems and demonstrate its capacity. Studies on natural language generation often employ sequence decoding, which generates the words comprising a sentence in sequential order and uses the input generated by each word in the previous step. In contrast, we propose a decoding method that employs a sequence generated by traversing a dependency tree with feed input to a pair consisting of a parent and sibling in the dependency tree. As a result, the most important words are generated first, thereby enabling words with greater relevance to be fed into the process. At prediction time, our model generates dependency trees and converts the trees into sentences. The proposed decoding method was evaluated by re-implementing a semantically controlled Long Short-Term Memory (SC-LSTM) structure for natural language generation, and the input and predicted sequence were converted to allow dependency tree decoding. The experimental results indicated that our suggested approach, i.e., dependency tree decoding, dramatically elevates the BLEU-score and naturalness. Furthermore, when creating sentences with n-best using dependency tree decoding, the word-diversity of the output sentences was increased by approximately 6%, offering a more diverse sentence pattern.

INDEX TERMS Machine learning, Multi-layer neural network, human computer interaction, natural language processing, dependency parsing, spoken dialog system

I. INTRODUCTION

The spoken dialog system (SDS) is a computer system that was designed to converse with users using speech and natural language, and plays a crucial role in human-machine interaction. As shown in Fig. 1, SDS is generally composed of automatic speech recognition (ASR), natural language understanding (NLU), dialog management (DM), natural language generation (NLG), and text-to-speech synthesis (TTS) [1][2]. NLG in SDS components generates natural language sentences that correspond to structured semantic representations. The semantic expression includes the intention and the information an SDS intends to convey. Thus, effective NLG dramatically improves the usability of human-machine interaction.

In recent years, various deep neural network (DNN) technologies have been applied to natural language processing tasks. Specifically, the recurrent neural network

(RNN) has demonstrated unequivocal advantages in tasks such as machine translation [3][4], image captioning [5][6], video captioning [7][8][9], video action recognition [10][11], video retrieval [12], information extraction [13][14], and NLG [15][16]. Among these applications, NLG is a particularly challenging task in an SDS. In this scenario, RNN-based approaches have not yet replaced the template-based approach because they do not provide stable performance. Template-based NLG provides reliable performance and is conceptually intuitive [17]. However, template-based NLG requires the construction of templates for each type of dialog act the dialog manager could potentially return as output. Furthermore, responses obtained from template-based NLG are constrained to the same type of dialog act as the input, which can be tedious for an SDS user. We suggest that the adaptation of DNN-based NLG to

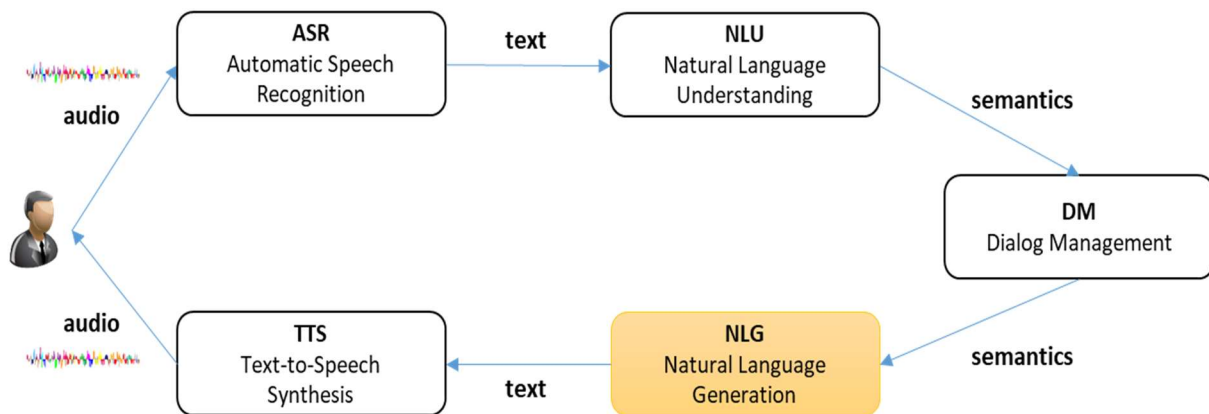


FIGURE 1. Overview of spoken dialog system.

provide stable performance would reduce the cost of template building and it would also increase the variety of sentences generated from the same dialog at the same time. Thus, DNN-based NLG could improve the quality and performance of the output in scenarios such as the SDS use case.

Currently, conventional DNN-based NLG usually employs a sequence decoding method that generates the words of a sentence in sequential order, moving forward or backward, using input generated by the word in the preceding step [18]. Sequence decoding is the simplest way to generate a sentence using RNN. However, an inherent disadvantage of this method is the possibility that, despite the likelihood of the words forming the input and output of the RNN cell being related, they may only be weakly correlated. Thus, in this pattern-rich environment, sequence decoding can degrade the performance.

In this paper, we propose dependency tree decoding as a novel decoding method capable of mitigating the disadvantages of sequence decoding. Dependency tree decoding constructs the words that form the input feed and those that form the output in the order in which the nodes are visited by the breadth-first search when constructing the dependency tree of the sentence. Then, the dependency tree can be obtained using a dependency parser. Dependency tree decoding has two advantages over sequence decoding. First, dependency tree decoding outputs words in the closest order, which promotes the tendency to output important words first. Second, dependency tree decoding connects the input word to parent and sibling words, which encourages a stronger correlation between the input feed and output word.

II. RELATED WORK

Template-based NLG is the most commonly applied approach in the field of NLG [17]. The main advantages of the template-based NLG method are the low number of errors and the ease with which developers can control the

system. However, because template-based NLG requires the definition of sentences for all possible dialog acts, the cost of the dialog system grows in proportion to its cost. Machine-learning-based NLG has attracted increasing attention as a potential solution to the constraints of template-based NLG. For example, in Bayesian networks-based NLG [19], new tags, referred to as semantic stacks, were defined by dividing a natural language sentence into phrase units. Then, a sentence was created using the Bayesian network. This method achieved improved performance even when annotating small datasets using active learning. However, existing research has primarily relied on a corpus consisting of only dialog acts and sentences because tagging the semantic stack is a relatively expensive annotation task.

Recurrent neural networks are another current focus in NLG research. One study [15] proposed a method to generate sentences that employed the one-hot encoded representation of the dialog act as the initial hidden state of the RNN in a manner similar to that of image captioning [5]. A subsequent study [18] improved the performance by proposing a semantically controlled Long Short-Term Memory (SC-LSTM) neural network structure to prevent the inclusion of words that have the same meaning or the inclusion of information that specific dialog act values are not present in sentences. Another study [20] demonstrated that a conditional variational auto-encoder offers good output with relatively less data in terms of the cross-domain. Then, Press *et al.* [21] showed that the generative adversarial network (GAN), known for its efficacy in image recognition, could also be employed for NLG.

Dependency parsing [22] analyzes syntactic structures such as dependency relations between words in a sentence. Dependency parsing is widely employed in natural language processing technologies such as opinion mining [23] and information extraction [24]. Deep neural networks have recently been an active research topic, mainly with respect to the implementation of a dependency tree on a sentence feature or RNN encoding. Sennrich and Barry [25]

demonstrated that using the parent label as a feature in a dependency tree effectively improves the machine translation (MT) performance. Furthermore, Tai *et al.* [26] encoded a tree-structured topology LSTM for instead of using sequence topology LSTM encoding, which improved the performance in terms of measuring the semantic relatedness as well as sentiment classification. Dependency parsing is also being actively used in encoding based on convolutional neural networks (CNNs). Yang *et al.* [27] applied a dependency tree to a position-encoding CNN, demonstrating that this implementation successfully provides semantic and syntactic representation in a relation extraction task. Other studies [28][29] suggested an encoder with an applied dependency tree in a graph convolutional network, demonstrating that semantic and syntactic information is well represented in both MT and semantic role labeling (SRL) tasks.

Most of these studies used dependency parsing either as an important feature to analyze sentences or as a method to encode sentences. In contrast, our study applies dependency parsing to the decoding step of sentence generation. The suggested method is especially characterized by its ability to create a dependency tree rather than sentences.

III. NATURAL LANGUAGE GENERATION USING DEPENDENCY TREE TRAVERSAL

Most existing NLG studies employ sequence decoding. However, the disadvantages of sequence decoding include a bottleneck in performance improvement, as described in Section I and II. To provide the input and output feeds, dependency tree decoding employs breadth-first search for the dependency tree instead of sequentially scanning a sentence.

A. PREPROCESSING

The proposed preprocessing method is schematically shown in Fig. 2. Preprocessing is executed using slot-value chunking, a sentence analyzer, dechunking, and delexicalization. Slot-value chunking simply converts the word sequence matched with the slot-value to the most appropriate word. Slot-value chunking is used to prevent problems caused by dependency parsing errors. For example, in the sentence of Fig. 2., if 'great' becomes a child of 'recommend' due to dependency parser error, delexicalization cannot work normally. The application of dependency tree decoding requires the dependency tree of a sentence to be constructed. To counteract the fact that most NLG corpora contain only sentences, and not dependency trees, we used sentence analyzers such as sentence segmentation, part-of-speech taggers, and dependency parsers, which are often required when using part-of-speech taggers. In our experiments, we employed the Google Cloud Natural Language Processing API [30][31] as the sentence analyzer because it provided all three of these functions. Next, we restored the chunked words and delexicalized for the slot-value.

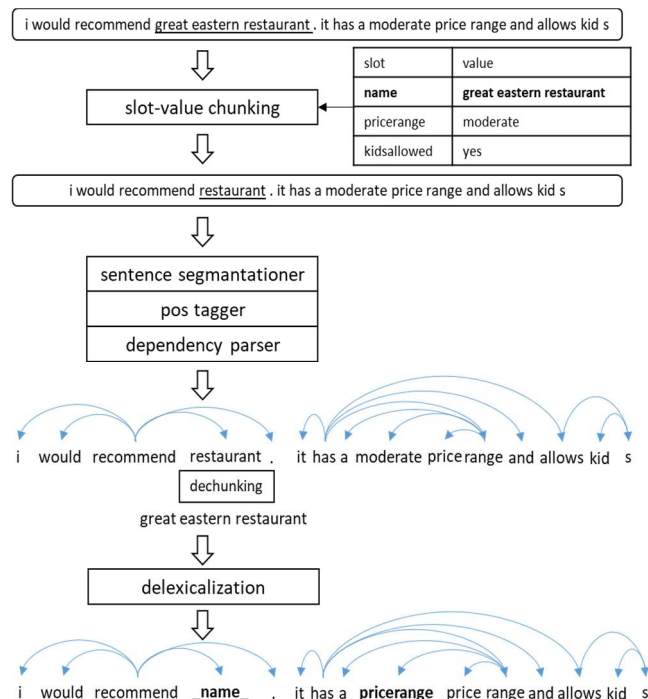


FIGURE 2. Preprocessing for dependency tree decoding: this is a necessary step to generate training data. The generated data include errors from the sentence analyzer, which may influence the performance of NLG.

B. DEPENDENCY TREES TO SEQUENCE

Learning and predicting using an RNN-based model requires us to sequentially feed the input and output; thus, it is not possible to feed the input and output directly into a dependency tree. In an attempt to overcome this problem, we employed a breadth-first search algorithm to convert the dependency trees into a sequence. For the purposes of the proposed algorithm, it was crucial that we were able to restore the elements of the generated sequence to the original trees. We defined several special markers to accomplish the restoration task, as detailed below:

- `_start_`, `_root_`: beginning of the tree;
- `_ls_`: start traversal of children to the left of the parent;
- `_le_`: traversal of the children to the left of the parent ended, and begin that of the children to the right;
- `_re_`: traversal of the children to the right of the parent ended;
- `_ended_`: no more trees.

Algorithm 1 provides the pseudocode of the breadth-first search procedure we modified to convert dependency trees into sequences. The inputs for the procedure are the dependency trees produced during preprocessing. Because

the input could contain multiple sentences, the routine is repeated from line 2 in tree2seq for each sentence. Although the tree2seq procedure resembles a common breadth-first algorithm, the proposed method is distinguished by the inserted lines that assign special markers to the input and output of the sequence. The procedure generates the children to the left in lines 9 to 16, and the children to the right in lines 17 to 23. Fig. 3 demonstrates the working flow of Algorithm 1 more intuitively. The upper part of Fig. 3 indicates the order in which each word is generated, which is the same as the traversal order of the breadth-first-search. Thus, dependency tree decoding generates the first word near the root on the dependency tree, resulting in the generation of a relatively more important word first in a sentence. The lower part of Fig. 3 demonstrates how the in/output of the RNN Cell is composed in the decoding process. As shown in the figure, the input of the RNN cell becomes the nearest sibling of the parent that was generated in the previous history, with the output being the child. The parent, sibling, and child are closely related to each other and this characteristic is an important feature, a second-order feature, in dependency parsing [32][33]. Consequently, the proposed method increases the relevance of the input and output of the RNN Cell compared to existing sequence decoding, enabling sentences to be generated more accurately. An example of the conversion from dependency trees to a sequence is shown in Fig. 4. However, the length of the sequences generated by Algorithm 1 was always three times the length of the corresponding input sentence. This last attribute could be problematic as increasing the length of the RNN could degrade performance by placing an increased burden on learning and prediction.

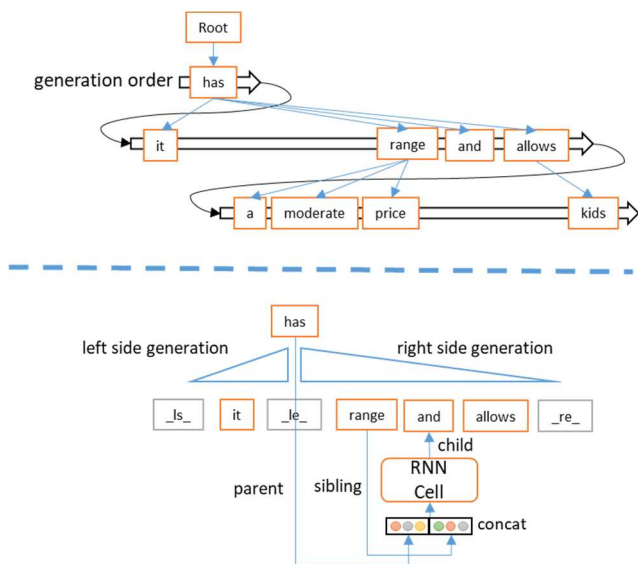


FIGURE 3. Example of dependency tree decoding process.

Algorithm 1 trees to sequence procedure

Input: tree_list = dependency tree list

Output: sequence = converted result containing input (parent, sibling) and output

trees2seq(tree_list)

```

1: sequence = []
2: For tree in tree_list:
3:   sequence.append(tree2seq(tree))
4: sequence.input.append('_start_', '_root_')
5: sequence.output.append('_ended_')
6: return sequence

```

tree2seq(tree)

```

1: sequence = []
2: sequence.input.append('_start_', '_root_')
3: sequence.output.append(tree.root)
4: queue = Queue()
5: queue.push(tree.root)
6: while queue is not empty:
7:   parent = queue.pop()
8:   sibling = '_ls_'
9:   for child in parent.left_child:
10:    queue.push(child)
11:   sequence.input.append((sibling, parent))
12:   sequence.output.append(child)
13:   sibling = child
14: sequence.input.append((sibling, parent))
15: sequence.output.append('_le_')
16: sibling = '_le_'
17: for child in parent.right_child:
18:   queue.push(child)
19:   sequence.input.append((sibling, parent))
20:   sequence.output.append(child)
21:   sibling = child
22: sequence.input.append((sibling, parent))
23: sequence.output.append('_re_')
24: return sequence

```

C. SEQUENCE TO DEPENDENCY TREES

At the time of word prediction, the RNN-based model that was trained using the dependency trees generates a sequence containing special markers. We then construct the dependency tree using the predicted sequence by using an approach similar to that of Algorithm 1. However, incorrect special marker predictions produced by the model may disturb the structure of the dependency tree. For instance, when the tree state is generating children to the left, i.e., lines 12 to 18 of Algorithm 1, the model can only generate `_le_` special markers. Additionally, if the model generates a slot-key, such as `__name__`, `__pricerange__`, that is not listed in

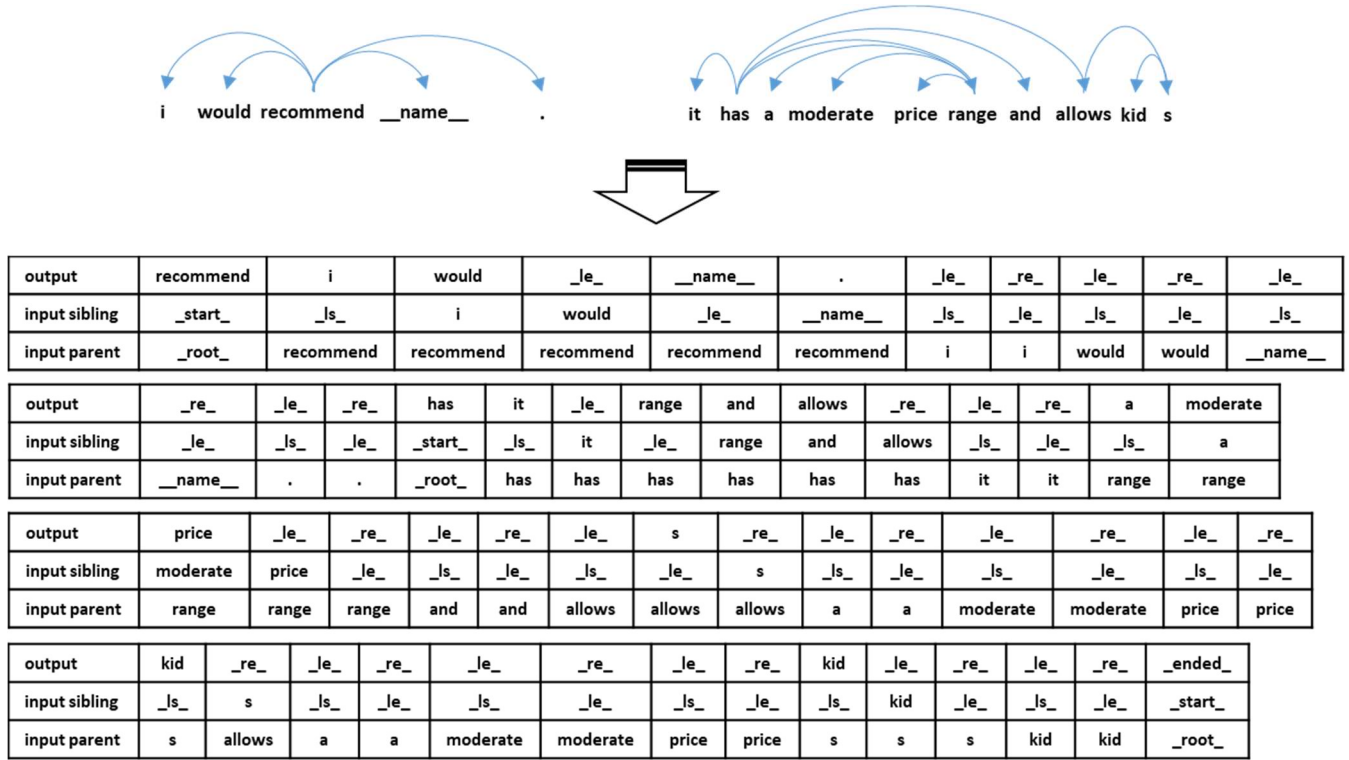


FIGURE 4. Example of dependency trees converted to sequences.

the input dialog act, then the generated sentences will be incorrect because the model would be unable to process the lexicalization. To avoid this situation, we defined the following constraint rules on the predictions:

- First prediction $\neq \{_ended_ ,_le_ ,_re_ \}$;
- Prediction of children to the left $\neq \{_ended_ ,_re_ \}$;
- Prediction of children to the right $\neq \{_ended_ ,_le_ \}$;
- First prediction of next sentence $\neq \{_le_ ,_re_ \}$;
- All prediction $\neq \{slot-keys\}$ unlisted in input dialog act, $_start_ ,_root_ ,_ls_ \}$.

According to the preceding rules, we excluded constrained special markers from the prediction candidates. The generated dependency trees are easily converted into sentences using the depth-first search algorithm. Finally, the sentences are completed by lexicalizing the slot-key to a value.

The dependency tree decoding process can employ beam search in a way similar to sequence decoding. Note that the tree, queue, and head must be copied and maintained separately when each state is expended during the beam search.

IV. SEMANTICALLY CONTROLLED LONG SHORT-TERM MEMORY (SC-LSTM)

The SC-LSTM neural network model [18] was employed to learn and predict for the sequenced dependency tree because this RNN-based model is well suited to time-sequence tasks. However, as the length of a sequence increases, it becomes increasingly difficult to process vanishing and exploding gradients. To overcome this limitation, a variant of LSTM, which adopts the concepts of a memory cell, and input, output, and forget gates, was proposed and successfully applied in various tasks. The conventional LSTM structure [34][35] is expressed as (1) to (7), where i_t is the input gate, o_t is the output gate, f_t is the forget gate and c_t is the memory cell at time t . The sigmoid function is represented by σ , $\tanh$ denotes the hyperbolic tangent, and $\odot$ is the elementwise multiplication operator. The input and forget gates determine the way in which the memory cell is updated using the input embedded word x_t and the hidden layer h_{t-1} at the current time t . The memory cell outputs the hidden layer h_t by the output gate. Finally, a word probability distribution is generated using the softmax function.

$$i_t = \sigma(W_{ix}x_t + W_{ih}h_{t-1}), \quad (1)$$

$$o_t = \sigma(W_{ox}x_t + W_{oh}h_{t-1}), \quad (2)$$

$$f_t = \sigma(W_{fx}x_t + W_{fh}h_{t-1}), \quad (3)$$

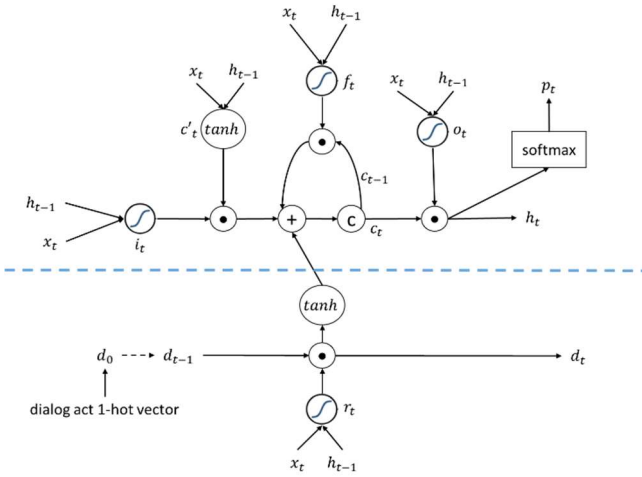


FIGURE 5. Semantically controlled Long Short-Term Memory (SC-LSTM) structure: the conventional LSTM appears above the dashed line, and the added DA-cell and control gate are depicted below it.

$$c'_t = \tanh(W_{cx}x_t + W_{ch}h_{t-1}), \quad (4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot c'_t, \quad (5)$$

$$h_t = o_t \odot c_t, \quad (6)$$

$$p_t = \text{Softmax}(h_t); \quad (7)$$

The SC-LSTM model contains an additional dialog act cell (DA-cell) and control gate, as compared to the conventional LSTM structure. The definitions of the DA-cell, control gate, and modified LSTM-cell are shown in (8) to (10), where r_t is the control gate, d_t is the DA-cell, and c_t is the modified LSTM-cell. The DA-cell memorizes the generation history as a semantic representation and conveys this information to the modified LSTM-cell. Then, the modified LSTM-cell reflects this information in the resulting prediction. The control gate regulates the generation history by reducing the features to be discarded in the DA-cell. The advantage of SC-LSTM is that it provides the generation history the semantics already generated and those not yet generated. Therefore, SC-LSTM prevents over-generation, repeated generation of the same semantics, as well as under-generation, or failure to generate the semantics. The structure of the SC-LSTM approach is depicted in Fig. 5.

$$r_t = \sigma(W_{rx}x_t + W_{rh}h_{t-1}), \quad (8)$$

$$d_t = r_t \odot d_{t-1}, \quad (9)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot c'_t + \tanh(W_{cd}d_t); \quad (10)$$

Similar to a conventional LSTM, an SC-LSTM may be stacked in multiple layers. As an SC-LSTM stacked in two layers outperforms a single-layer SC-LSTM [18], we applied a two-layer SC-LSTM to dependency tree decoding.

Stacking the SC-LSTM required (11) to be adjusted to the definition of the control gate, where l is the hidden layer index and α_l is the weight value for each layer.

$$r_t = \sigma(W_{rx}x_t + \sum_l \alpha_l w_{rh}^l h_{t-1}^l); \quad (11)$$

V. EXPERIMENTS

A. EXPERIMENTAL SETUP

The experiments that were used to evaluate the proposed SC-LSTM network were conducted on a spoken dialog system corpus used to suggest venues in San Francisco [36]. We used the Restaurant and Hotel domains in the corpus. The statistics for each dataset are provided in Table I. These two domains have similar objects of conversation, the sizes of their datasets are similar, average length of sentence, and they have similar semantic tag types. The Google Cloud Natural Language Processing API was used to analyze the sentences of the corpus, and the dataset was reconstructed as described in Section III-B.

TABLE I
DATASET STATISTICS

Domain	Restaurant	Hotel
Train/Valid/Test	3114/1039/1039	3223/1075/1075
dialog acts	reqmore, goodbye, select, confirm, request, inform, inform_only, inform_count, inform_no_match	
shared slots	name, type, area, near, price, phone, address, postcode, pricerange	
specific slots	food, goodformel, kids-allowed	hasinternet, acceptscards, dogs-allowed
average length of sentence (delexicalized)	7.64	7.69
average length of sentence	9.01	9.28

We evaluated the dependency tree decoding by re-implemented the SC-LSTM model using TensorFlow 1.7 [37]. The hyper-parameters of the re-implemented SC-LSTM are listed in Table II. To pre-train the embedding of words, we applied GloVe word2vec [38] as pre-trained by Wikipedia 2014 + Gigaword 5 [39]. The model was trained by optimizing the cross-entropy cost function via Adam [40]. The learning rate was reduced by half if the total cost in the validation set did not decrease for five epochs. Model training was executed iteratively until the learning rate was less than 10^{-5} . We also applied dropout to input x and the LSTM-cell to prevent over-fitting of the model [41], [42]. We randomly shuffled the corpus for each training epoch to prevent the model from over-fitting the rearward data in the corpus. The baselines applied in the experiments for SC-LSTM with sequence decoding may be downloaded from

GitHub [36]. Our model and the baseline model used the same dialog act feature vector, but we used the hyper-parameters of the baseline.

TABLE II
HYPER-PARAMETERS

Parameter	Value
word embedding size	80
learning rate	10^{-3}
batch size	16
learning rate decay	0.5
decay epoch	5
minimum learning rate	10^{-5}
LSTM hidden size	80
dropout rate	0.5
weight value l	[0.5, 0.5]

We evaluated the performance of the proposed model using the bilingual evaluation understudy algorithm BLEU-4 [43] and human evaluation. BLEU-4 metric is defined in (12), where $prec_n$ is the n -gram precision between the predicted and reference sentences.

$$BLEU_4 = \min(1, \frac{out.leng}{ref.leng}) \times (\prod_{n=1}^4 prec_n^{\frac{1}{4}}); \quad (12)$$

Human evaluation was conducted by averaging the scores given by two English language majors to 50 randomly selected data from the test corpus. The evaluators applied two scoring metrics, informativeness and naturalness, each on a scale of 1 to 3, with 3 being the optimum.

B. RESULTS AND ANALYSIS

The results of the BLEU-4 evaluation are provided in Table III. In the results, the beam property is the width of the beam search. Further, top-1 property is the result of the sentence with the highest score of the generated candidates, and top-5 averages the five highest scores of the candidates. These results demonstrate that although the proposed model, with dependency tree decoding, provides an overall improvement in performance, certain cases did exhibit lower performance. Considering the results for each domain, the overall performance improved in the restaurant domain, and equal performance is provided in the hotel domain. Two factors could have reduced the performance of the dependency tree decoding model. The first factor is error propagation within the sentence analyzer. The dependency trees of the training corpus were generated with the errors of the sentence analyzer, thus errors from that point onward make it more difficult for the model to learn the pattern. The second factor is that the length of the RNN was increased compared to other methods. As discussed in Section III-B, the way the model uses dependency tree decoding was a substantial burden with respect to learning patterns, because the RNN was three times longer than usual. Nevertheless, the model

based on dependency tree decoding achieved an overall improvement in performance. We suggest that this improvement can be extended by further development of the sentence analyzer and RNN model.

TABLE III
EXPERIMENTAL RESULTS OF BLEU-4 SCORE

Decoding	Restaurant	Hotel
sequence decoding (beam=1, top=1)	77.94	88.49
sequence decoding (beam=5, top=1)	77.92	89.70
sequence decoding (beam=10, top=1)	77.98	89.88
dependency tree decoding (beam=1, top=1)	82.20	90.80
dependency tree decoding (beam=5, top=1)	81.32	91.98
dependency tree decoding (beam=10, top=1)	81.48	91.92
sequence decoding (beam=5, top=5)	74.00	84.14
sequence decoding (beam=10, top=5)	73.65	84.77
dependency tree decoding (beam=5, top=5)	75.07	83.82
dependency tree decoding (beam=10, top=5)	75.58	83.45

The results of human evaluation are detailed in Table IV. Note that the model that employed dependency tree decoding improved the naturalness while reducing the informativeness compared to the model that used conventional sequence decoding. However, reduction of the informativeness can be avoided as the informativeness of the output of top-1 offers almost the same performance. Closer examination of these results highlights several key features. The first is that sentences generated by dependency tree decoding more often exhibit under-generation for the slot-value, as seen in the first example provided in Table V. In this example, the 1st, 2nd, 4th, and 5th-ranked results indicate that the model with dependency tree decoding did not generate slot semantics in a one-by-one manner. This resulted in the low informativeness values. The second feature is that the sentences that were produced using sequence decoding were more likely to have an incorrect meaning, as also illustrated by the first example in Table V. In this example, there could be a misunderstanding related to the fact that “near glen park” modifies the meaning of “kid” in this context. This may be because the proposed model represents the grammar of the dependency tree more effectively. The third feature of these results is that sentences that were produced using sequence decoding more often underwent over-generation, as illustrated by the second and third examples in Table V. Finally, in very rare cases, dependency tree decoding produces the exact same sentence for the top n candidates. This phenomenon occurs when the generated trees are different, but the converted sentences are the same.

TABLE IV
HUMAN EVALUATION OF INFORMATIVENESS AND NATURALNESS

Decoding		Restaurant	Hotel	Average
sequence decoding (beam=5, top=1)	inform.	2.95	2.93	2.94
	natural.	2.69	2.75	2.72
sequence decoding (beam=5, top=5)	inform.	2.91	2.97	2.94
	natural.	2.66	2.68	2.67
dependency tree decoding (beam=5, top=1)	inform.	2.90	3.00	2.95
	natural.	2.78	2.91	2.84
dependency tree decoding (beam=5, top=5)	inform.	2.83	2.95	2.89
	natural.	2.76	2.88	2.85

C. WORD DIVERSITY

Key characteristics related to word diversity may be observed in the generated sentences. Sentences produced using dependency tree decoding exhibited more diverse patterns. One of the primary objectives of NLG is to increase the variety in sentences generated for the same dialog act. The third example in Table V reveals that the sentences produced using dependency tree decoding exhibited greater variation in their sentence pattern than those produced using sequence decoding. The reason for this phenomenon is that the branching of sentence patterns occurs near the root, when sentences are generated with a beam search. To measure this effect more precisely, we defined the Word Diversity (WD) metric as (13), where n is the number of sentences generated.

$$WD = \frac{\text{size of word set of generated sentences}}{\sum_{i=1}^n \text{num of word in sentence}_i} \quad (13)$$

WD is the number of types of words appearing in a sentence divided by the total number of words. Thus, a higher WD value indicates greater variety. The results of WD evaluation are provided in Table VI. These results confirm that the proposed dependency tree decoding method realizes substantial improvement in WD.

TABLE VI
EXPERIMENTAL RESULTS OF WORD DIVERSITY

Decoding	Restaurant	Hotel
sequence decoding (beam=5, top 5)	0.274	0.265
dependency tree decoding (beam=5, top 5)	0.349	0.314

D. CROSS-DOMAIN EVALUATION

We conducted experiments using the same dataset as [20] to verify that dependency tree decoding works well in a cross-

domain environment. The metric used in experiments is BLEU-4, and the results are provided in Table VII. The results demonstrate that the proposed method, with dependency tree decoding, provides an improvement in performance on dataset of restaurant and hotel domains as compared to sequence decoding. Though it is difficult to make a direct performance comparison with SCVAE (Semantically Conditioned Variational Autoencoder) [20] because of different experimental environment, the results suggest that the proposed method provides improved performance compared to the existing methods on restaurant and hotel domains. On the other hand, the results on laptop and TV domains demonstrate that the performance of the proposed method provides lower performance than sequence decoding and SCVAE. The laptop and TV domains consist of more complex sentences as the average sentence of the data is approximately twice longer than restaurant and hotel domains (laptop is 23.42, TV is 21.44). The accuracy of dependency parser and recurrent neural network model tends to decrease as the sentence length increases. The experimental results show that these data characteristics may be significant disadvantages for the proposed method because of dependency parsing errors and issues with long RNNs. Therefore, the proposed method should be used considering the environment of the target spoken dialog systems. In addition, we can prevent the performance degradation of the proposed model with a more improved dependency parser and neural network model in the future.

TABLE VII
EXPERIMENTAL RESULTS ON CROSS-DOMAIN

Domain	sequence decoding (beam=5, top 5)	dependency tree decoding (beam=5, top 5)	SCVAE [20]
restaurant	0.713	0.774	0.540
Hotel	0.819	0.827	0.652
Laptop	0.506	0.452	0.478
TV	0.480	0.413	0.442

VI. CONCLUSION

In this paper, we proposed the use of dependency tree decoding to generate natural language. The proposed method generates a dependency tree instead of a sentence in the decoding process. Dependency tree decoding improved the correlation between the words that were used as input and those that were predicted and it generated the words near the root node earlier in the process. The experimental results demonstrated that the proposed model generates sentences that were improved in terms of over-generation, naturalness, and diversity.

In the future, we intend to address the problem of under-generation. Another important task is to extend the proposed dependency tree decoding method to enable it to perform other tasks, such as machine translation and image captioning, effectively.

TABLE V
EXAMPLES OF SENTENCES PRODUCED BY NATURAL LANGUAGE
GENERATION (NLG) (BEAM=5, TOP 5)

Dialog act: inform(name = 'chenery park'; near = 'glen park'; kidsallowed = yes; pricerange = moderate)
with sequence decoding
chenery park is a moderate -ly priced restaurant that allows kid -s and is near glen park
chenery park is a moderate -ly priced restaurant that allows kid -s near glen park
chenery park is a moderate -ly priced restaurant that allows kid -s near the glen park area
chenery park is a moderate restaurant near glen park that allows child -s
chenery park is a moderate -ly priced restaurant that allows kid -s and is near the glen park area
with dependency tree decoding
chenery park is a moderate ly priced restaurant that allows kid s
chenery park is a moderate ly priced restaurant that allows child s
chenery park is a moderate restaurant near glen park that allows child s
chenery park is a moderate ly priced restaurant that allows kid s
chenery park is a nice restaurant near glen park that allows kid s
Dialog act: goodbye()
with sequence decoding
Goodbye
thank you for calling. good bye
thank you, goodbye
thank you for calling. thank you for calling. good bye
thank you for calling. thank you for calling. thank you for calling. good bye
with dependency tree decoding
Goodbye
good bye
thank you, goodbye
Bye
thank you for calling. Goodbye
Dialog act: ?confirm(pricerange = 'pricey')
with sequence decoding
are you looking for something in the pricey price range
are you looking for something in the pricey price range you are looking for a hotel in
are you looking for a pricey hotel
are you looking for something in the pricey price range you are looking for a hotel in the you range
are you looking for something in the pricey price range you are looking for a hotel near
with dependency tree decoding
would you like a pricey hotel
do you want a pricey hotel
can i confirm that you are looking for a pricey hotel
can i confirm that you are looking for something in the pricey price range
so, you are looking for something in the pricey price range

REFERENCES

- [1] S. J. Young, "Probabilistic methods in spoken-dialogue systems," in *Philosophical Transactions of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*, vol. 358(1769), 2000, pp. 1389-1402.
- [2] W. Xu, and A. I. Rudnicky, "Task-based dialog management using an agenda," In *Proc. 2000 ANLP/NAACL Workshop on Conversational systems*, 2000, pp. 42-47.
- [3] K. Cho, B. V. Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations

- using rnn encoder-decoder for statistical machine translation," In *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process.* (EMNLP 2014), Doha, Qatar, 2014, pp. 1724-1734.
- [4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," In *Proc. Int. Conf. Learn. Represent.* (ICLR 2015), San Diego, California, 2015.
- [5] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," In *Proc. Conf. Comput. Vis. Patt. Recog.* (CVPR 2015), 2015.
- [6] P. Anderson, B. Fernando, M. Johnson, S. Gould, "Guided open vocabulary image captioning with constrained beam search," In *Proc. 2017 Conf. Empir. Methods Nat. Lang. Process.* (EMNLP 2017), Copenhagen, Denmark, 2017, pp. 936-945.
- [7] A. Dilawari, M. U. G. Khan, A. Farooq, Z. U. Rehman, S. Rho and I. Mehmood, "Natural language description of video streams using task-Specific feature encoding," in *IEEE Access*, vol. 6, 2018, pp. 16639-16645.
- [8] J. Song, Y. Guo, L. Gao, X. Li, A. Hanjalic, and H. T. Shen, "From deterministic to generative: multimodal stochastic rnns for video captioning. in *IEEE transactions on neural networks and learning systems*, vol. 99, 2018, pp1-12.
- [9] L. Gao, Z. Guo, H. Zhang, X. Xu, and H. T. Shen, "Video captioning with attention-based lstm and semantic consistency," in *IEEE Transactions on Multimedia*, vol. 19(9), 2018, pp. 2045-2055.
- [10] W. Wang, L. Gao, J. Song, and H. Shen, Beyond frame-level cnn: saliency-aware 3-d cnn with lstm for video action recognition," in *IEEE Signal Processing Letters*, vol. 24(4), 2017, pp. 510-514.
- [11] W. Wang, L. Gao, R. Wang, X. Sun, and X. Liu, "Two-stream 3-D convnet fusion for action recognition in videos with arbitrary size and length," in *IEEE Transactions on Multimedia*, vol. 20(3), 2018, pp. 634-644.
- [12] J. Song, H. Zhang, X. Li, L. Gao, M. Wang, and R. Hong, "Self-supervised video hashing with hierarchical binary auto-encoder," in *IEEE Transactions on Image Processing*, vol. 27(7), 2018, pp. 3210-3221.
- [13] W. Zeng, J. Tang and X. Zhao, "Entity linking on chinese microblogs via deep neural network," in *IEEE Access*, vol. 6, 2018, pp. 25908-25920
- [14] X. Luo, W. Zhou, W. Wang, Y. Zhu and J. Deng, "Attention-based relation extraction with bidirectional gated recurrent unit and highway network in the analysis of geological data," in *IEEE Access*, vol. 6, 2018, pp. 5705-5715.
- [15] T. Wen, M. Gašić, D. Kim, N. Mrkšić, P. Su, D. Vandyke, and S. Young, "Stochastic language generation in dialogue using recurrent neural networks with convolutional sentence reranking," In *Proc. 16th Annu. Meet. Spec. Int. Group Discourse Dial. (SIGDIAL 2015)*, Prague, Czech Republic, 2015.
- [16] D. Ondrej, and F. Jurčiček, "Sequence-to-sequence generation for spoken dialogue via deep syntax trees and strings," In *Proc. 54th Annu. Meet. Assoc. Comput. Ling. (ACL 2016)*, Berlin, Germany, 2016, pp. 45-51.
- [17] S. Busemann, and H. Horacek, "A flexible shallow approach to text generation," In *Proc. Int. Nat. Lang. Generat. Workshop*, Niagara-on-the-Lake, USA, 1998, pp. 238-247.
- [18] T. Wen, M. Gašić, N. Mrkšić, P. Su, D. Vandyke, and S. Young, "Semantically conditioned lstm-based natural language generation for spoken dialogue systems," In *Proc. 2015 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2015)*, Lisbon, Portugal, 2015, pp. 1711-1721.
- [19] F. Mairesse, M. Gašić, F. Jurčiček, S. Keizer, B. Thomson, K. Yu, and S. Young, "Phrase-based statistical language generation using graphical models and active learning," In *Proc. 48th Annu. Meeting Assoc. Comput. Ling.*, Uppsala, Sweden, 2010, pp 1552-1561.
- [20] B. H. Tseng, F. Kreyssig, P. Budzianowski, P., I. Casanueva, Y. C. Wu, S. Ultes, and M. Gašić, Variational cross-domain natural language generation for spoken dialogue systems." In *Proc. 19th Annu. Meet. Spec. Int. Group Discourse Dial. (SIGDIAL 2018)*, Melbourne, Australia, 2018, pp. 338-343.
- [21] O. Press, A. Bar, B. Bogin, J. Berant, and L. Wolf, "Language generation with recurrent generative adversarial networks without pre-training," In *Proc. 1st Workshop Learn. Generate Nat. Lang. ICML 2017*, Sydney, Australia, 2017.

- [22] M. A. Covington, "A dependency parser for variable-word-order languages," Research Report AI-1990-01, University of Georgia, 1990.
- [23] D. McClosky, M. Surdeanu, and C. D. Manning, "Event extraction as dependency parsing," In *Proc. 49th Annu. Meet. Assoc. Comput. Ling.: Human Lang. Technol. (HLT 2011)*, 2011, pp. 1626-1635.
- [24] L. D. Caro, and M. Grella, "Sentiment analysis via dependency parsing," *Comput. Stand. Interf.*, vol. 35, no. 5, pp. 442-453, 2013.
- [25] R. Sennrich, and H. Barry, "Linguistic input features improve neural machine translation," In *Proc. 1st Conf. on Machine Translation: Volume 1, Research Papers*, vol. 1, 2016, pp. 83-91.
- [26] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," In *Proc. 53rd Annu. Meet. Assoc. Comput. Ling. (ACL 2015) and 7th Int. Joint Conf. on Nat. Lang. Processing (IJCNLP 2015)*, Beijing, China, 2015, pp. 26-31.
- [27] Y. Yang, Y. Tong, S. Ma, and Z. H. Deng, Z. H., "A position encoding convolutional neural network based on dependency tree for relation classification," In *Proc. 2016 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2016)*, 2016, pp. 65-74.
- [28] J. Bastings, I. Titov, W. Aziz, D. Marcheggiani, and K. Simaan, "Graph convolutional encoders for syntax-aware neural machine translation," In *Proc. 2017 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2017)*, 2017, pp. 1957-1967.
- [29] D. Marcheggiani, and I. Titov, "Encoding sentences with graph convolutional networks for semantic role labeling," In *Proc. 2017 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2017)*, 2017, pp. 1506-1515.
- [30] D. Andor, C. Alberti, D. Weiss, A. Severyn, A. Presta, K. Ganchev, S. Petrov, and M. Collins, "Globally normalized transition-based neural networks," In *Proc. 54th Annu. Meet. Assoc. Comput. Ling. (ACL 2016)*, Berlin, Germany, 2016, pp. 2442-2452.
- [31] <https://cloud.google.com/natural-language/>
- [32] R. McDonald, F. Pereira, "Online learning of approximate dependency parsing algorithms," In *Proc. 11th Conf. Eur. Chapter Assoc. Comput. Ling. (EACL 2006)*, 2006, pp. 81-88.
- [33] T. Koo, and M. Collins, "Efficient third-order dependency parsers," In *Proc. 48th Annu. Meet. Assoc. Comput. Ling. (ACL 2010)*, 2010, pp. 1-11.
- [34] S. Hochreiter, and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, 1997.
- [35] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," In *Proc. 38th IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP 2013)*, 2013.
- [36] <https://github.com/shawnwun/RNNLG>
- [37] <https://www.tensorflow.org/>
- [38] J. Pennington, R. Socher, C. D. Manning, "Glove: Global vectors for word representation," In *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2014)*, Doha, Qatar, 2014, pp. 1532-1543.
- [39] <https://nlp.stanford.edu/projects/glove/>
- [40] D. P. Kingma, J. Ba, "Adam: A method for stochastic optimization," In *Proc. Int. Conf. Learn. Represent. (ICLR 2015)*, San Diego, California, 2014.
- [41] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, 15(1), pp. 1929-1958, 2014.
- [42] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," arXiv preprint arXiv:1409.2329, 2014.
- [43] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "BLEU: A method for automatic evaluation of machine translation," In *Proc. 40th Annu. Meet. Assoc. Comput. Ling. (ACL 2002)*, 2002, pp. 311-318.